

*Informe de Trabajo Práctico: Desarrollo de
Videojuego con Entorno Gráfico Java*

**UNIVERSIDAD NACIONAL DE GENERAL
SARMIENTO**

**Técnico Universitario en Informática /
Licenciatura en Sistemas**

Materia: Programación 1

Fecha de Entrega: 9 de Junio 2026

INTEGRANTES DEL EQUIPO

Nombre y Apellido	Dirección de Email
Calderón Gonzalo	gonzalocal127@gmail.com
Ayala Matias	mmattias0105@gmail.com
Díaz Maximiliano	Diazmaximilianonahuel33@gmail.com

1. Introducción

Este trabajo práctico consistió en el diseño y desarrollo de un videojuego interactivo en 2D utilizando el lenguaje **Java** y la biblioteca gráfica **Entorno**. El objetivo principal fue aplicar los fundamentos de la Programación Orientada a Objetos (POO), tales como la encapsulación, modularidad y la gestión de colecciones de datos.

El juego es un plataformas que consta de dos niveles:

Nivel 1: El jugador controla a una heroína (**Princesa**) que debe avanzar por un mapa con plataformas, esquivar pozos y eliminar enemigos hasta llegar al **Castillo**.

Nivel 2: Se activa una batalla contra un **Jefe** final que ataca mediante proyectiles orbitales expansivos programados con lógica trigonométrica.

2. Descripción General del Sistema

El motor del juego se ha estructurado dividiendo el dominio del problema en clases distintas.

A continuación se desglosa la composición de cada componente lógico del sistema informático:

2.1. Clase Juego

Propósito: Actúa como el punto de entrada ejecutable de la aplicación informática y el puente directo con la ventana del subsistema gráfico. Hereda de Interface Juego para integrarse con los ticks de tiempo continuos.

Variables de Instancia:

entorno: Instancia contenedora que inicializa la ventana del lienzo gráfico (800x600 píxeles) y gestiona el bucle de renderizado periódico.

niveles: Controlador lógico general que encapsula los estados de las escenas, aislando las transiciones del método raíz.

Descripción de Métodos:

Juego(): Configura la ventana gráfica con títulos explícitos, dimensiona los límites de resolución, instala el gestor de niveles e inicia el hilo principal del motor gráfico.

tick(): Se ejecuta en bucle indefinido a razón de sesenta iteraciones por segundo. Delega en su totalidad la responsabilidad de actualización cromática y posicional al gestor lógico interno.

main(String[] args): Método de inicio de la máquina virtual que lanza la ejecución del juego instanciando de forma anónima la raíz.

2.2. Clase Niveles

Propósito: Centraliza el estado del juego, administrando la instancia concurrente de los

elementos interactivos de cada nivel y orquestando el orden secuencial de actualizaciones lógicas y renderizado en pantalla.

Variables de Instancia principales: Contiene referencias a la princesa, el castillo, el proyectil principal, el gestor de plataformas (plataformas), el coordinador de amenazas (enemigos), la entidad del jefe, y cuatro instancias especializadas de proyectiles orbitales (jefeProyectil1..4). Además, regula el factor de desplazamiento horizontal de la vista en píxeles mediante `camarax` e identifica el escenario actual con `nivel`.

Descripción de Métodos:

`inicializarNivel1()` e `inicializarNivel2()`: Métodos de configuración que actúan como fábricas de objetos, limpiando colecciones previas y disponiendo las coordenadas físicas correspondientes a cada estado de simulación.

`actualizarNivel()`: Evaluador central del bucle que bifurca el procesamiento interno invocando los sub-métodos de ejecución según el entero del nivel activo, contemplando asimismo las pantallas finales de Game Over o Victoria.

`ejecutarNivel1()` y `ejecutarNivel2()`: Contienen la secuencia estricta de llamadas de comportamiento: movimiento de actores, actualización de temporizadores de inmunidad, cálculo estricto de colisiones, refresco de coordenadas y llamadas a primitivas gráficas de dibujo.

2.3. Clase Princesa

Propósito: Modela el avatar del jugador en la simulación, hicimos el movimiento en las direcciones solicitadas en la consigna, tales como inercia horizontal, salto y caída.

Variables de Instancia principales: Coordenadas central (x, y), (ancho, alto), (velocidadX, velocidadY), contadores enteros de salud (vidas) y una variable clave de enfriamiento temporal (`tiempoInvulnerable`).

Descripción de Métodos:

`moverIzquierda()`

`moverDerecha()`

`salto()`

`gravedad()`: Le resta el valor en Y a la princesa para que haga la ilusión de que cae una vez llega a la máxima altura en el salto.

2.4. Clase Plataforma

Propósito: Representa los bloques geométricos sólidos sobre los cuales colisionan las entidades móviles del mapa. Pueden poseer comportamiento de soporte estructural o actuar como marcadores lógicos de pozos vacíos.

Variables de Instancia: Coordenadas absolutas de eje central (x, y), dimensiones (ancho, alto) y la bandera lógica condicional de peligro (`esPozo`).

Descripción de Métodos:

`dibujo(double camarax)`: Renderiza un rectángulo relleno azulado restando el desfase de la cámara horizontal para emular un scrolling lateral fluido.

`ColisionConPrincesa(Princesa p)`: Si detecta superposición entre rectángulos alineados, evalúa las profundidades de intrusión relativa en X e Y para discernir el sentido de colisión.

Si es vertical superior, restablece la altura de la princesa apoyándola en el borde y anulando su inercia.

Si es lateral, desplaza el personaje hacia afuera para bloquear su avance contra paredes.

2.5. Clase Proyectoil

Propósito: Representa la herramienta ofensiva que equipa el jugador para defenderse de los peligros de la escena. Cuenta con dos estados claros: transporte pasivo y vuelo balístico independiente.

Variables de Instancia: Posición (x, y), magnitud de velocidad traslacional constante, componentes de dirección vector x, y (vx, vy) y booleanos (disparado, equipado).

Descripción de Métodos:

actualizar(Princesa p): Si el proyectil no se ha disparado, modifica sus coordenadas para posicionarse de forma relativa sobre las manos del avatar. Al activarse el disparo, conmuta a movimiento lineal uniforme guiado por el vector de dirección.

disparar(int mouseX, int mouseY): Calcula la diferencia posicional en ambos ejes cartesianos desde el origen del disparo hacia las coordenadas de clic del ratón, obtiene la hipotenusa mediante la norma euclidiana y descompone el vector para asegurar velocidad lineal uniforme en 360 grados.

disparo(Princesa p, Entorno e): Enlace de ciclo de vida que evalúa de forma integrada el renderizado, el sentido de recolección en el suelo y el clic del mouse para ejecutar la eyección balística.

2.6. Clase Jefe

Propósito de Jefe: Controla el comportamiento cíclico del antagonista principal en el Nivel 2. Realiza movimientos lineales horizontales automáticos y expone una zona de impacto reactiva capaz de reflejar los proyectiles que le arrojan.

Propósito de JefeProyectoil: Estructura el comportamiento del ataque coordinado del jefe mediante el modelado matemático de órbitas circulares que se expanden y contraen dinámicamente.

Variables de Instancia (JefeProyectoil):

x, y: Coordenadas calculadas a partir del centro orbital.

alto, ancho: Dimensiones métricas del proyectil de color naranja (10x10 píxeles).

grados: Ángulo actual en el sistema sexagesimal que determina la rotación sobre la órbita.

radio: Distancia escalar en píxeles medida respecto al punto pivote central.

radioRadio: Factor cinético diferencial que expande o contrae el radio orbital en cada ciclo temporal.

Descripción de Métodos (JefeProyectoil):

calcularAngulo(double centrox, double centroY): Convierte el valor angular de grados sexagesimales a radianes matemáticos, aplicando funciones circulares trigonométricas directas (Math.cos y Math.sin) multiplicadas por la longitud del radio actual para proyectar y actualizar la posición absoluta en el espacio bidimensional.

`moverProyectil(double centroX, double centroY)`: Incrementa de forma fija el componente angular en 5 grados sexagesimales por tick para producir la rotación orbital y ejecuta un ciclo de reinicio para evitar el desborde numérico una vez superado el umbral de una revolución completa (360 grados). Actualiza la posición física invocando el cálculo trigonométrico.

`animacionRadio()`: Modifica el escalar de distancia sumando algebraicamente el incremento de `radioRadio`. Cuenta con límites estrictos de control (200 y 40 píxeles) que actúan como umbrales elásticos, invirtiendo el signo algebraico de la velocidad radial para originar un patrón visual armónico de expansión y contracción oscilatoria continua.

`dibujarJefeProyectil()`: Ejecuta el llamado gráfico sobre la interfaz de dibujo para pintar la figura geométrica naranja con su respectivo desfase angular nulo.

2.7. Clases de Soporte (Castillo, Enemigo, Gestionadores)

Justificación del Diseño: Uso de Clases Gestionadoras vs. Archivo Único

La decisión de implementar `GestionadorPlataformas` y `GestionadorEnemigos` en lugar de acumular toda la lógica dentro de una única clase central (como `Juego` o `Niveles`) responde a principios y buenas prácticas de la Programación Orientada a Objetos:

- 1) Si hubiéramos colocado el código en un solo archivo, esta clase habría tenido que encargarse simultáneamente de renderizar la pantalla, leer el teclado, calcular las físicas de la princesa, actualizar los enemigos y controlar los vectores de las plataformas.

Al separar los gestionadores, cada clase tiene una única razón para cambiar; por ejemplo, si se modifica la forma en que aparecen los enemigos, solo se edita el `GestionadorEnemigos`, sin riesgo de romper el sistema de colisiones del mapa.

- 2) Meter todo en un mismo archivo genera "código espagueti", donde las variables globales se entrelazan de forma compleja.
- 3) Control de errores, se manejan múltiples entidades dinámicas que aparecen y desaparecen (como enemigos que mueren o proyectiles que se destruyen),
- 4) Los gestionadores actúan como una capa de protección (abstracción), realizando comprobaciones estrictas de existencia previa antes de ejecutar cualquier operación física o gráfica.

Abstracciones Menores:

Castillo: Representa la meta física de rescate. Cuenta con un disparador por tiempo (120 ticks) que requiere que el jugador permanezca inmóvil encima del bloque para validar las condiciones de victoria y cambio de etapa.

Enemigo: Abstracción de peligro básico con desplazamiento rectilíneo predecible de izquierda a derecha.}

GestionadorPlataformas y GestionadorEnemigos: Implementan contenedores que encapsulan vectores estáticos y proveen métodos de utilidad masiva (crearPiso, colisionesPlataformas, actualizarEnemigos).

Esto permite iterar de manera limpia protegiendo al motor de excepciones de puntero nulo gracias a comprobaciones estrictas de existencia previa.

Problemas Encontrados, Decisiones y Soluciones Técnicas

1. Muerte instantánea por ráfagas de daño (Bugs de colisión repetida)

Problema: Al chocar con un enemigo, el juego restaba vidas en cada *frame* sucesivo, vaciando la salud de la Princesa en menos de una décima de segundo.

Solución: Se añadió la variable **tiempoInvulnerable** (60 *frames*) en la clase **Princesa**. Al recibir daño, pierde una vida y se activa este contador; mientras sea mayor a cero, se ignoran nuevas colisiones y el personaje parpadea visualmente para dar un tiempo de recuperación.

2. Vidas del jefe y problemas con la colisión del proyectil:

Problema: Al crear el nivel 2 las vidas se salían de la pantalla y además cuando el proyectil colisiona con el jefe al detectar múltiples colisiones, se moría el jefe en el primer disparo.

Solución: Al jefe detectar la colisión con el proyectil devuelve un booleano que dentro del archivo niveles.java hace que el proyectil se vuelva null, perdiendo 1 sola vida.

Y las vidas al pasar de 7 la variable x que determina la posición en pantalla se suma con 20 para volver a 0 y a Y se le suma 40, provocando que este una línea por debajo de la barra ya hecha con las vidas.

3. Caída infinita y pérdida de referencia del suelo

Problema: Si la Princesa caía en un pozo o caminaba al vacío desde una plataforma flotante, el sistema no detectaba la falta de soporte, fallando al activar la gravedad y que pierda vida y vuelva al lugar en el que estaba antes.

Solución: Se rediseñó la lógica en **GestionadorPlataformas** con un contador de intersecciones. Si tras recorrer todas las plataformas el contador es cero, se asume que la heroína está en el aire, cambiando el estado a **setEnElSuelo(false)** para activar la caída libre.

4. Lógica de proyectiles y colisiones con enemigos

Problema: Gestionar de forma eficiente múltiples colisiones entre los proyectiles del jugador y la lista de enemigos dinámicos.

Solución: Se delegó el control en **GestionadorEnemigos**. Mediante bucles limpios, se evalúan las cajas de colisión. Si un proyectil activo interseca a un enemigo, ambos se marcan para ser eliminados del mapa y destruidos en el siguiente ciclo físico. Osea se vuelven null y se eliminan.

5. Sistema de Vidas y fin del juego

Problema: Controlar el flujo de transiciones de pantalla (*Game Over* y *Victoria*) de manera sincronizada con el estado de salud de las entidades principales.

Solución: Se centralizaron los contadores numéricos en el gestor de niveles. Si la salud de la Princesa llega a cero, el bucle interrumpe la simulación y salta directo a la pantalla de derrota. Por el contrario, si las vidas del Jefe llegan a cero en el nivel 2, se valida la condición de victoria general.

6. Dinámica del Trabajo Grupal

Problema: La integración de código desarrollado por distintos miembros generaba conflictos de versiones, superposición de funciones en el método principal y dificultades para acoplar las clases de física con el entorno gráfico.

Solución: Se adoptó una división de tareas basada en ramas.

Se asignó el diseño de las entidades autónomas a una rama y los sistemas de colisiones y control de escenas a otra rama.

El uso de interfaces y firmas de métodos predefinidos permite integrar todo el código en la clase **Niveles** con un mínimo de fricción y fallos de merge.

3. Implementación (Código fuente)

Link:

<https://github.com/Gonzalocal1/progra1Tp.git>

4. Conclusiones y Lecciones Aprendidas

Como conclusión nunca hicimos antes algo parecido, fue un gran reto, más con el trabajo virtual y coordinar en llamadas, pero pudimos mejorar nuestras habilidades técnicas y personales para seguir mejorando en la carrera.

Fue un arduo trabajo pero estamos contentos con el resultado obtenido, esperamos que la nota nos ayude mucho para terminar la cursada.

Gracias por leer.